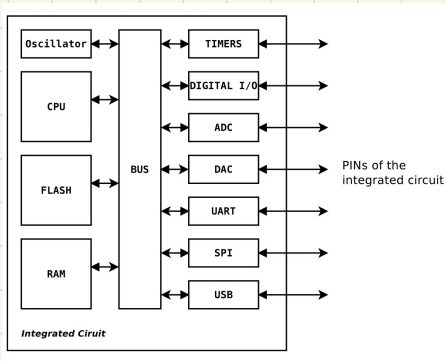




# Introduzione

Un microcontrollore è un circuito integrato che include tutte le parti di un computer completo. In particolare include **cpu**, **l'oscillatore per il clock**, **una memoria flash (ROM)**, **RAM** e **periferiche I/O**.  
I pin sono collegati direttamente alle periferiche I/O.



Sono usati nelle auto, negli elettrodomestici, nelle pompe di benzina, nel contesto IOT etc...  
Generalmente non hanno un S.O., o ne hanno uno piccolo con funzionalità minime.  
Quando ne programmiamo uno senza S.O., dobbiamo programmare anche le sue periferiche I/O.

## Typical MCU design pattern

```
#include "..."  
  
/* global variables */  
  
int main()  
{  
    /* initialization of peripherals */  
  
    /* infinite loop */  
    for (;;) {  
        /* (wait events) */  
        /* read inputs */  
        /* process data */  
        /* write outputs */  
    }  
  
    /* Interrupt Handlers for peripherals */  
    void xxx_IRQHandler(void)  
    {  
        ...  
    }  
}
```

Una certa regione della memoria è riservata per le periferiche di I/O e in questa regione ogni locazione di memoria ha un significato specifico. Queste locazioni di memoria sono chiamate **Special Function Register (SFRs)**.  
Scrivere dentro un SFR significa programmare una specifica periferica I/O.

Per farlo useremo delle variabili globali grazie ad una libreria fatta ad hoc.

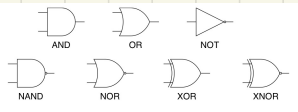
# Circuiti Logici e Segnali

## Circuiti Logici

I circuiti logici sono caratterizzati dal fatto che il voltaggio dei fili può assumere solo 2 valori: 0 (0V) o 1 (+VDD, la potenza dell'intero circuito).  
Si possono dividere in 3 categorie: Combinatori, Combinatori speciali e sequenziali.

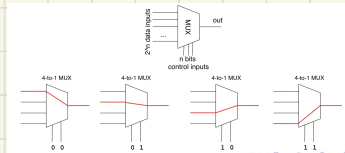
### Combinatori

- L'output dipende solo dallo stato corrente dell'input
- Un esempio sono le porte logiche
- Il loro comportamento è rappresentato da una tabella di verità



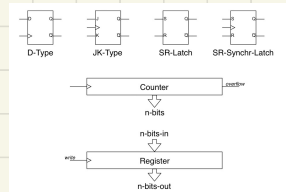
### Combinatori Speciali

- Multiplexer: agisce come uno switch di segnale
- Un dato input alla volta è instradato nell'output in base a dei bit di controllo.



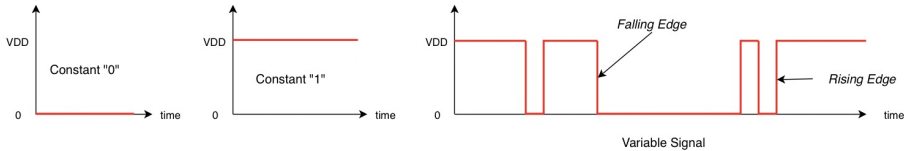
### Sequenziali

- Sono ad esempio flip-flop, contatori e registri.
- L'output dipende dallo stato corrente e passato dell'input: sono circuiti logici con memoria
- Sono sensibili alle variazioni dell'input e il loro comportamento è rappresentato da una macchina a stati finiti

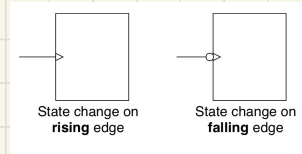


# Segnali Logici

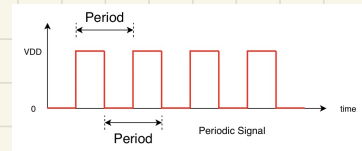
I segnali logici possono essere costanti o variabili. Quelli costanti sono stati logici che non cambiano nel tempo, quelli variabili possono cambiare e sono caratterizzati da angoli di **salita** (quando lo stato cambia da "0" a "1"), e di **salita** (quando il segnale cambia da "1" a "0").



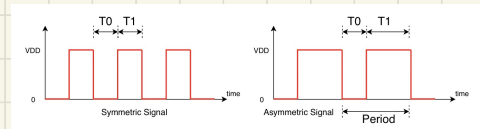
I circuiti sequenziali sono sensibili agli angoli e sono rappresentati con "triangoli".



I segnali variabili possono essere periodici quando la distanza di tempo tra angoli dello stesso tipo è costante e questa distanza è chiamata **periodo (P)**.



Inoltre, i segnali periodici possono essere simmetrici o asimmetrici, dipende se il tempo in cui lo stato è "1" è lo stesso di quando lo stato è "0" oppure no. La percentuale del tempo in cui lo stato è "1" si chiama "ciclo di lavoro".



## Connessione Hardware-Software

Un sensore di presenza genera una pulsazione ogni volta che una periferica viene collegata. Questo sensore è collegato ad un contatore e ogni volta che il sensore genera una pulsazione, il contatore aumenta il proprio valore.

### Reading/Writing the Counter

```
...
int32_t * counter_ptr = (int32_t *)0x80c000;
...

/* clearing the counter */
*counter_ptr = 0;
...

/* printing the counter */
printf("Counter value %d\n", *counter_ptr);
```

### Configuring the Counter

```
int32_t * config_ptr = (int32_t *)0x80c004;

/* counter input from Internal Oscillator */
*config_ptr = 0;

/* counter input from External Counter Input */
*config_ptr = 0x40;
```

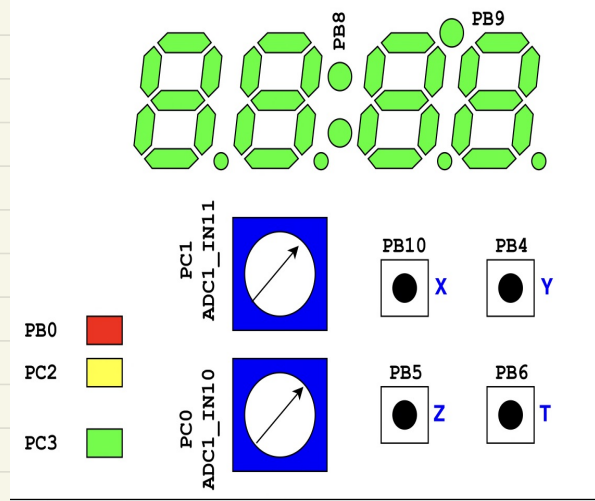
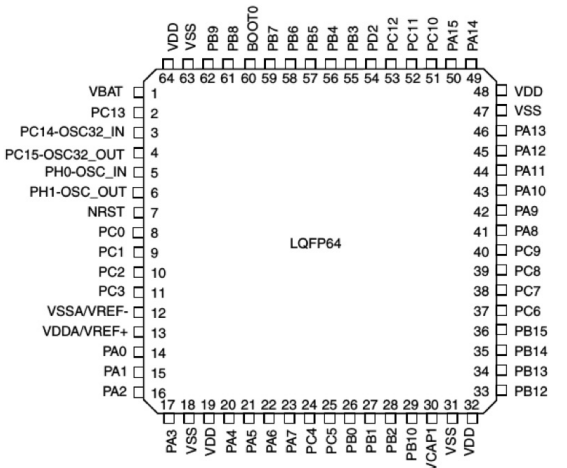
I circuiti di un microcontrollore infatti sono connessi al software grazie a registri e contatori. I registri sono mappati in memoria con degli indirizzi di memoria e l'interazione tra hardware e software avviene leggendo e scrivendo in questi indirizzi di memoria.

# L'interfaccia digitale I/O

È un'interfaccia in cui ogni pin elettrico può avere due stati, 0 o 1, e può essere configurato come input o come output.

Nel nostro microcontrollore abbiamo varie porte, ognuna delle quali ha 16 bit, corrispondenti a 16 pin. Ogni pin è rappresentato come "Px<sub>y</sub>", dove x è la porta e y il numero del pin.

Figure 12. STM32F401xD/xE LQFP64 pinout



Per usare uno specifico pin di una porta (GPIO), dobbiamo:

- 1) Inizializzare la porta;
- 2) Specificare se considerare il pin come input o output;
- 3) Leggere nel pin se è considerato come input o scrivere nel pin se è considerato come output.

## Illuminare un led

```
#include "stm32_unict_lib.h"

int main()
{
    // initialize port B
    GPIO_init(GPIOB);

    // configure pin PB0 as output
    GPIO_config_output(GPIOB, 0);

    // infinite loop
    for (;;) {
        GPIO_write(GPIOB, 0, 1); // set PB0 to 1
        delay_ms(500);           // wait 0.5 secs
        GPIO_write(GPIOB, 0, 0); // set PB0 to 0
        delay_ms(500);           // wait 0.5 secs
    }
}
```

Premere un bottone e  
accendere un led

( GPIO è una variabile globale,  
un puntatore a GPIO\_TypeDef che  
rappresenta l'indirizzo di memoria  
dell'SFR dello port )

```
#include "stm32_unict_lib.h"

int main()
{
    // pushbutton on PB10; LED on PC2

    // initialize ports
    GPIO_init(GPIOB);
    GPIO_init(GPIOC);

    // configure pin PB10 as input
    GPIO_config_input(GPIOB, 10);

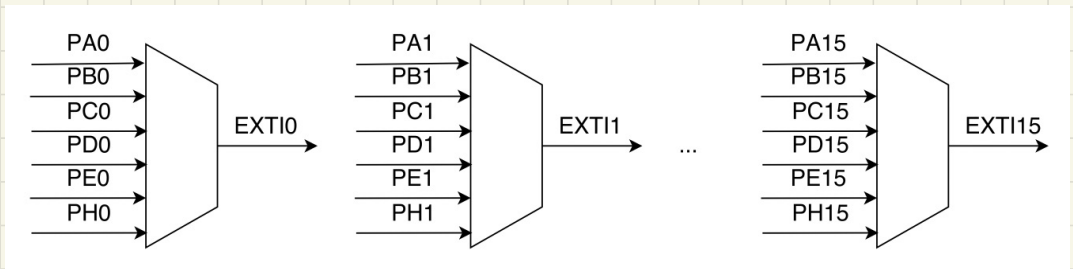
    // configure pin PC2 as output
    GPIO_config_output(GPIOC, 2);

    // infinite loop
    for (;;) {
        int pinval = GPIO_read(GPIOB, 10);
        GPIO_write(GPIOC, 2, !pinval);
    }
}
```

# External Interrupt

Ogni pin può far innescare un IRQ (Interrupt Request) e l'IRQ può essere configurato per innescarsi al verificarsi di un angolo di salito, di un angolo di caduta o di entrambi gli angoli. Il circuito che gestisce la configurazione si chiama External Interrupt Controller (EXTI).

EXTI gestisce 23 linee di interrupt (EXTI0, EXTI1... EXTI22): dalla 0 alla 15 possono essere usate per le porte GPIO, dalla 16 alla 22 sono invece collegate internamente ad altre periferiche. Il tutto è gestito da un multiplexer, che consente il collegamento di una linea di interrupt ad una sola porta GPIO al numero di pin corrispondente.



Per usare un interrupt dobbiamo:

- 1) Collegare una linea EXTI ad una porta GPIO;
- 2) Configurare la linea EXTI per gestire uno specifico segnale;
- 3) Scrivere l'Interrupt Service Routine: qui dobbiamo scrivere le operazioni da svolgere e poi cancellarle, il tutto con specifiche funzioni;

Premere un bottone e far lampeggiare un led

```
#include "stm32_unict_lib.h"

int main()
{
    // LED at PC3
    GPIO_init(GPIOC);
    GPIO_config_output(GPIOC, 3);

    // pushbutton Y (PB4)
    GPIO_init(GPIOB);
    GPIO_config_input(GPIOB, 4);

    GPIO_config_EXTI(GPIOB, EXTI4);
    EXTI_enable(EXTI4, FALLING_EDGE);

    for (;;) { } // do nothing
}

void EXTI4_IRQHandler(void)
{
    if (EXTI_isset(EXTI4)) {
        GPIO_toggle(GPIOC, 3);
        EXTI_clear(EXTI4);
    }
}
```

# Il Display

Il Display è composto da 4 segmenti e può essere usato per mostrare sia numeri che lettere. I 4 segmenti sono numerati da 0 a 3.

Utilizza il timer TIM5, quindi questo timer non può essere usato quando il display è attivato.

- Display initialization  
`void DISPLAY_init(void)`
- Write a character "c" in digit "d"  
`void DISPLAY_putc(int d, char c)`
- Write a string "s" from digit "d"  
`void DISPLAY_puts(int digit, char * s)`
- Turn on or off the decimal point of digit "d"  
`void DISPLAY_dp(int digit, int on_off)`

## Counter di 10 secondi su display

```
#include "stm32_unict_lib.h"
#include <stdio.h>

int counter = 0;

int main()
{
    // Init the display
    DISPLAY_init();

    // init the timer 2 and configure the timebase
    // tick set to 0.1 ms
    TIM_init(TIM2);
    TIM_config_timebase(TIM2, 8400, 1000);
    TIM_set(TIM2, 0);
    TIM_on(TIM2);
    TIM_enable_irq(TIM2, IRQ_UPDATE);

    for (;;) {
        char s[5];
        sprintf(s, "%4d", counter); // put counter value to string
        DISPLAY_puts(0, s); // update the display
    }
}

void TIM2_IRQHandler(void)
{
    ++counter;
    TIM_update_clear(TIM2);
}
```

# I timers

Un timer è fondamentalmente un circuito che abilita il software ad avere una "consapevolezza del tempo"; è fondamentalmente una variabile globale che aumenta e decrementa in base ad una sorgente di clock programmabile.

L'hardware del timer è composto da 3 parti programmabili:

- La sorgente di clock, che genera il tic tac del timer;
- La base del tempo, da cui deriva la "granularità" del tempo;
- Il circuito "slave", che fornisce delle funzionalità aggiuntive, come la misurazione della pulsazione o la generazione di segnali.

Il microcontrollore STM32 offre 11 timer, ma la nostra libreria ne supporta 5: TIM1, TIM2, TIM3, TIM4, TIM5 (che è usato dal display)

Il conteggio è partito dalla base del tempo con i seguenti registri:

- $TIMx \rightarrow PSC$ : il "prescaler register", che determina il fattore di divisione;
- $TIMx \rightarrow CNT$ : il "counter register", che incrementa o decrementa il valore del conteggio;
- $TIMx \rightarrow ARR$ : l'"auto-reload register", che obbliga CNT a contare da 0 ad ARR per poi resettarsi. Quando CNT arriva ad ARR, genera un segnale che può essere usato per attivare un interrupt.

- Initialize a TIMER:

```
void TIM_init(TIM_TypeDef * timer);
```

- Configure the timebase:

```
void TIM_config_timebase(TIM_TypeDef * timer,  
                          uint16_t prescaler,  
                          uint16_t autoreload);
```

- Start a timer:

```
void TIM_on(TIM_TypeDef * timer);
```

- Stop a timer:

```
void TIM_off(TIM_TypeDef * timer);
```

- Read the counter:

```
int16_t TIM_get(TIM_TypeDef * timer);
```

- Write the counter:

```
void TIM_set(TIM_TypeDef * timer, int16_t value);
```

- Check if an update event occurred:

```
int TIM_update_check(TIM_TypeDef * timer);
```

- Clears the update event notification:

```
void TIM_update_clear(TIM_TypeDef * timer);
```

Per settare il clock a 1ms, dobbiamo avere un fattore di divisione di 8400.

## Far lampeggiare un led usando un timer

```
#include "stm32_unict_lib.h"

int main()
{
    // LED at PC3
    GPIO_init(GPIOC);
    GPIO_config_output(GPIOC, 3);

    // init the timer
    TIM_init(TIM2);

    // Configure the timebase
    // Counter clock set to 0.1 ms
    TIM_config_timebase(TIM2, 8400, 5000);

    TIM_set(TIM2, 0); // resets the counter
    TIM_on(TIM2); // starts the timer

    // infinite loop
    for (;;) {
        // check the update event
        if (TIM_update_check(TIM2)) {
            GPIO_toggle(GPIOC, 3);
            // clear the update event
            TIM_update_clear(TIM2);
        }
    }
}
```

## Far lampeggiare un led usando un bottone

```
#include "stm32_unict_lib.h"

int main()
{
    int last_key_state, flashing = 0;
    // LED at PC3
    GPIO_init(GPIOC);
    GPIO_config_output(GPIOC, 3);

    // pushbutton X (PB10)
    GPIO_init(GPIOB);
    GPIO_config_input(GPIOB, 10);

    TIM_init(TIM2); // init the timer

    // Configure the timebase, counter clock set to 0.1 ms
    TIM_config_timebase(TIM2, 8400, 5000);
    TIM_set(TIM2, 0); // resets the counter
    TIM_on(TIM2); // starts the timer

    last_key_state = GPIO_read(GPIOB, 10);
    for (;;) {
        int current_key_state = GPIO_read(GPIOB, 10);
        if (last_key_state == 1 && current_key_state == 0) flashing = !flashing;
        last_key_state = current_key_state;
        if (TIM_update_check(TIM2)) {
            if (flashing) GPIO_toggle(GPIOC, 3);
            else GPIO_write(GPIOC, 3, 0);
            TIM_update_clear(TIM2);
        }
    }
}
```

## Fa lampeggiare un led usando gli interrupt

```
#include "stm32_unict_lib.h"

int flashing = 0;

int main()
{
    int last_key_state;
    // LED at PC3
    GPIO_init(GPIOC);
    GPIO_config_output(GPIOC, 3);
    // pushbutton X (PB10)
    GPIO_init(GPIOB);
    GPIO_config_input(GPIOB, 10);

    // init the timer
    TIM_init(TIM2);

    // Configure the timebase
    // Counter clock set to 0.1 ms
    TIM_config_timebase(TIM2, 8400, 2500);

    TIM_enable_irq(TIM2, IRQ_UPDATE);
    TIM_set(TIM2, 0); // resets the counter
    TIM_on(TIM2); // starts the timer

    last_key_state = GPIO_read(GPIOB, 10);
    for (;;) {
        int current_key_state = GPIO_read(GPIOB, 10);
        if (last_key_state == 1 && current_key_state == 0) flashing = !flashing;
        last_key_state = current_key_state;
    }
}

void TIM2_IRQHandler(void)
{
    if (flashing) GPIO_toggle(GPIOC, 3);
    else GPIO_write(GPIOC, 3, 0);
    TIM_update_clear(TIM2);
}
```

# Il convertitore Analogico-Digitale (ADC)

L'ADC è un circuito che prende in input un segnale analogico (in Volt) e lo converte in un segnale digitale, quindi in una variabile intera proporzionale al segnale ricevuto. L'ADC lavora considerando un input che va in un certo range, che va da  $V_{ref-}$  a  $V_{ref+}$ . Se l'input è  $V_{ref-}$ , l'ADCVAL sarà uguale a 0, se l'input è  $V_{ref+}$  l'ADCVAL sarà uguale a  $2^{NBITS} - 1$ .

$$ADCVAL = \left[ (V_{in} - V_{ref-}) \frac{2^{NBITS} - 1}{V_{ref+} - V_{ref-}} \right]$$

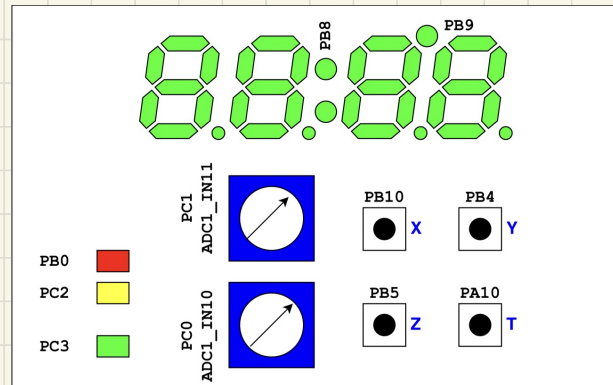
nel nostro caso ↴

$$ADCVAL = \left[ V_{in} \frac{2^{NBITS} - 1}{3.3} \right]$$

L'ADC ha vari canali di input (nel nostro caso solo ADC1), ma solo uno può essere selezionato alla volta, poiché collegati ad un multiplexer. Esso lavora seguendo 3 passi:

- Campionamento del segnale;
- Conversione del segnale;
- Fine della conversione del segnale;

| Pin | Analog Channel |  | Pin | Analog Channel |
|-----|----------------|--|-----|----------------|
| PA0 | ADC1_IN0       |  | PA1 | ADC1_IN1       |
| PA2 | ADC1_IN2       |  | PA3 | ADC1_IN3       |
| PA4 | ADC1_IN4       |  | PA5 | ADC1_IN5       |
| PA6 | ADC1_IN6       |  | PA7 | ADC1_IN7       |
| PB0 | ADC1_IN8       |  | PB1 | ADC1_IN9       |
| PC0 | ADC1_IN10      |  | PC1 | ADC1_IN11      |
| PC2 | ADC1_IN12      |  | PC3 | ADC1_IN13      |
| PC4 | ADC1_IN14      |  | PC5 | ADC1_IN15      |



Il nostro microcontrollore ha 4 risoluzioni configurabili (che determinano il valore massimo che può assumere l'ADCVAL, il valore del segnale una volta convertito):

- 6 bits [0,63]
- 8 bits [0,255]
- 10 bits [0,1023]
- 12 bits [0,4095]

Il risultato della conversione può essere allineato a sinistra o a destra rispetto a una variabile da 16 bits:

#### 12bit Left-Aligned

|     |     |    |    |    |    |    |    |    |    |    |    |   |   |   |   |
|-----|-----|----|----|----|----|----|----|----|----|----|----|---|---|---|---|
| b11 | b10 | b9 | b8 | b7 | b6 | b5 | b4 | b3 | b2 | b1 | b0 | 0 | 0 | 0 | 0 |
|-----|-----|----|----|----|----|----|----|----|----|----|----|---|---|---|---|

#### 12bit Right-Aligned

|   |   |   |   |     |     |    |    |    |    |    |    |    |    |    |    |
|---|---|---|---|-----|-----|----|----|----|----|----|----|----|----|----|----|
| 0 | 0 | 0 | 0 | b11 | b10 | b9 | b8 | b7 | b6 | b5 | b4 | b3 | b2 | b1 | b0 |
|---|---|---|---|-----|-----|----|----|----|----|----|----|----|----|----|----|

## Funzioni per l'ADC

#### Initialize an ADC:

```
void ADC_init(ADC_TypeDef * adc, int res, int align);
```

- **adc**, the ADC circuit
- **res**, the resolution in bits
  - **ADC\_RES\_6**
  - **ADC\_RES\_8**
  - **ADC\_RES\_10**
  - **ADC\_RES\_12**
- **align**, the bit alignment
  - **ADC\_ALIGN\_RIGHT**
  - **ADC\_ALIGN\_LEFT**

#### Configure the input(s):

```
void ADC_channel_config(ADC_TypeDef * adc,  
                        GPIO_TypeDef * port,  
                        int pin, int chan);
```

- **adc**, the ADC circuit
- **port**, the GPIO port of the input
- **pin**, the GPIO pin of the input
- **chan**, the ADC channel associated to the input

#### Start an ADC circuit:

```
void ADC_on(ADC_TypeDef * adc);
```

#### Stop an ADC circuit:

```
void ADC_off(ADC_TypeDef * adc);
```

#### Select a channel to convert:

```
void ADC_sample_channel(ADC_TypeDef * adc, int chan);
```

- **adc**, the ADC circuit
- **chan**, the ADC channel to be converted

#### Start a sample+conversion of the selected channel:

```
void ADC_start(ADC_TypeDef * adc);
```

#### Check if a conversion has been completed:

```
int ADC_completed(ADC_TypeDef * adc);
```

#### Read the converted value:

```
int ADC_read(ADC_TypeDef * adc);
```

## Convertire il Segnale e mostrare il valore

```
#include <stdio.h>
#include "stm32_unict_lib.h"

void main(void)
{
    DISPLAY_init();

    ADC_init(ADC1, ADC_RES_8, ADC_ALIGN_RIGHT);
    ADC_channel_config(ADC1, GPIOC, 0, 10);
    ADC_on(ADC1);
    ADC_sample_channel(ADC1, 10);

    for (;;) {
        ADC_start(ADC1);
        while (!ADC_completed(ADC1)) {}

        int value = ADC_read(ADC1);
        char s[4];
        sprintf(s, "%4d", value);
        DISPLAY_puts(0,s);
    }
}
```

## Fare lampeggiare un led con un periodo variabile

Vogliamo fare lampeggiare un led con un periodo che va da 50 a 500 ms. Quindi settiamo la timebase del timer a 0,5ms e l'autoreload dovrà essere tra 100 e 1000. Possiamo usare la formula:

$$ARR = ADCVAL \frac{1000 - 100}{255} + 100$$

```
#include <stdio.h>
#include "stm32_unict_lib.h"

int new_arr_value = 100;

void main(void)
{
    DISPLAY_init();
    GPIO_init(GPIOB);    GPIO_config_output(GPIOB, 0);

    ADC_init(ADC1, ADC_RES_8, ADC_ALIGN_RIGHT);
    ADC_channel_config(ADC1, GPIOC, 0, 10);
    ADC_on(ADC1); ADC_sample_channel(ADC1, 10);

    TIM_init(TIM2);
    TIM_config_timebase(TIM2, 42000, 100);
    TIM_set(TIM2, 0); TIM_enable_irq(TIM2, IRQ_UPDATE);
    TIM_on(TIM2);

    for (;;) {
        ADC_start(ADC1);
        while (!ADC_completed(ADC1)) {}
        int value = ADC_read(ADC1);
        new_arr_value = value * 900/255 + 100;
        char s[4];
        sprintf(s, "%4d", new_arr_value / 2); // we will display the milliseconds
        DISPLAY_puts(0,s);
    }
}
```

```
void TIM2_IRQHandler(void)
{
    if (TIM_update_check(TIM2)) {
        GPIO_toggle(GPIOB, 0);
        TIM_update_clear(TIM2);
        TIM2->ARR = new_arr_value;
        // update the autoreload register with new value
    }
}
```

# Operazioni Bitwise

Per eseguire delle operazioni sugli Special Function Register abbiamo bisogno di manipolare i singoli bit, con operazioni del tipo:

- Impostare a 1 un bit;
- Cancellare (impostare a 0) un bit;
- Invertire un bit;
- Testare un bit.

Sono tutte operazioni che si eseguono usando una "bit-mask", un pattern di bit.

**Impostare un bit:** Si fa un'operazione **OR** con il bit in questione posto a "1" e tutti gli altri a "0".

• Example: setting the bit 3 of the (8-bit) variable A:

| A    | B7 | B6 | B5 | B4 | B3 | B2 | B1 | B0 | OR |
|------|----|----|----|----|----|----|----|----|----|
| mask | 0  | 0  | 0  | 0  | 1  | 0  | 0  | 0  | =  |
| A    | B7 | B6 | B5 | B4 | 1  | B2 | B1 | B0 |    |

$A = A \mid 0x08;$

$A |= 0x08;$

**Cancellare un bit:** Si fa un'operazione

**AND** con il bit in questione posto a "0" e tutti

gli altri a "1".

• Example: clearing the bit 6 of the (8-bit) variable A:

| A    | B7 | B6 | B5 | B4 | B3 | B2 | B1 | B0 | AND |
|------|----|----|----|----|----|----|----|----|-----|
| mask | 1  | 0  | 1  | 1  | 1  | 1  | 1  | 1  | =   |
| A    | B7 | 0  | B5 | B4 | B3 | B2 | B1 | B0 |     |

$A = A \& 0xbf;$

$A \&= 0xbf;$

**Cancellare un bit:** Si fa un'operazione  
(con la negazione)

gli altri a "1".

**AND** con il bit in questione posto a "0" e tutti

• Example: clearing the bit 6 of the (8-bit) variable A:

$A = A \& \sim 0x40;$

$A \&= \sim 0x40;$

| B7 | B6 | B5 | B4 | B3 | B2 | B1 | B0 |   |
|----|----|----|----|----|----|----|----|---|
| ~  | 0  | 1  | 0  | 0  | 0  | 0  | 0  | = |
|    | 1  | 0  | 1  | 1  | 1  | 1  | 1  |   |

| A    | B7 | B6 | B5 | B4 | B3 | B2 | B1 | B0 | AND |
|------|----|----|----|----|----|----|----|----|-----|
| mask | 1  | 0  | 1  | 1  | 1  | 1  | 1  | 1  | =   |
| A    | B7 | 0  | B5 | B4 | B3 | B2 | B1 | B0 |     |

Invertire un bit: Si fa un'operazione  
gli altri a "0".

XOR con il bit in questione posto a "1" e tutti

- Example: toggling the bit 4 of the (8-bit) variable A:

| A    | B7 | B6 | B5 | B4 | B3 | B2 | B1 | B0 | XOR |
|------|----|----|----|----|----|----|----|----|-----|
| mask | 0  | 0  | 0  | 1  | 0  | 0  | 0  | 0  | =   |
| A    | B7 | B6 | B5 | B4 | B3 | B2 | B1 | B0 |     |

$A = A \oplus 0x10;$

$A \wedge = 0x10;$

Testare un bit: Si fa un'operazione AND con il bit in questione posto a "1" e tutti  
gli altri a "0".

- Check if the result is zero or non-zero
- Example: testing the bit 5 of the (8-bit) variable A:

| A    | B7 | B6 | B5 | B4 | B3 | B2 | B1 | B0 | AND |
|------|----|----|----|----|----|----|----|----|-----|
| mask | 0  | 0  | 1  | 0  | 0  | 0  | 0  | 0  | =   |
|      | 0  | 0  | B5 | 0  | 0  | 0  | 0  | 0  |     |

```
if ((A & 0x20) != 0) ... // non-zero
```

Esempio:

We want to set the MODE of PIN12 of GPIOA as "output":

#### 8.4.1 GPIO port mode register (GPIOx\_MODER) (x = A..E and H)

Address offset: 0x00

Reset values:

- 0x0C00 0000 for port A
- 0x0000 0280 for port B
- 0x0000 0000 for other ports

|              |              |              |              |              |              |             |             |             |             |             |             |             |             |             |             |
|--------------|--------------|--------------|--------------|--------------|--------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|
| 31           | 30           | 29           | 28           | 27           | 26           | 25          | 24          | 23          | 22          | 21          | 20          | 19          | 18          | 17          | 16          |
| MODER15[1:0] | MODER14[1:0] | MODER13[1:0] | MODER12[1:0] | MODER11[1:0] | MODER10[1:0] | MODER9[1:0] | MODER8[1:0] | MODER7[1:0] | MODER6[1:0] | MODER5[1:0] | MODER4[1:0] | MODER3[1:0] | MODER2[1:0] | MODER1[1:0] | MODER0[1:0] |
| rw           | rw           | rw           | rw           | rw           | rw           | rw          | rw          | rw          | rw          | rw          | rw          | rw          | rw          | rw          | rw          |
| 15           | 14           | 13           | 12           | 11           | 10           | 9           | 8           | 7           | 6           | 5           | 4           | 3           | 2           | 1           | 0           |
| MODER7[1:0]  | MODER6[1:0]  | MODER5[1:0]  | MODER4[1:0]  | MODER3[1:0]  | MODER2[1:0]  | MODER1[1:0] | MODER0[1:0] |             |             |             |             |             |             |             |             |
| rw           | rw           | rw           | rw           | rw           | rw           | rw          | rw          |             |             |             |             |             |             |             |             |

Bits 2y:2y+1 **MODERy[1:0]**: Port x configuration bits (y = 0..15)

These bits are written by software to configure the I/O direction mode.

00: Input (reset state)

01: General purpose output mode

10: Alternate function mode

11: Analog mode

1 Clear bits 24-25: **GPIOA->MODER &= 0xfcffffff;**

2 Set bit 24: **GPIOA->MODER |= 0x01000000;**

Possiamo fare queste operazioni anche con lo shift dei bit, con tanti shift quante è il numero bit da manipolare. Lo shift è a sinistra.

We want to set the MODE of PIN12 of GPIOA as “output”:

#### 8.4.1 GPIO port mode register (GPIOx\_MODER) (x = A..E and H)

Address offset: 0x00

Reset values:

- 0x0C00 0000 for port A
- 0x0000 0280 for port B
- 0x0000 0000 for other ports

|              |    |              |    |              |    |              |    |              |    |              |    |             |    |             |    |
|--------------|----|--------------|----|--------------|----|--------------|----|--------------|----|--------------|----|-------------|----|-------------|----|
| 31           | 30 | 29           | 28 | 27           | 26 | 25           | 24 | 23           | 22 | 21           | 20 | 19          | 18 | 17          | 16 |
| MODER15[1:0] |    | MODER14[1:0] |    | MODER13[1:0] |    | MODER12[1:0] |    | MODER11[1:0] |    | MODER10[1:0] |    | MODER9[1:0] |    | MODER8[1:0] |    |
| rw           | rw | rw           | rw | rw           | rw | rw           | rw | rw           | rw | rw           | rw | rw          | rw | rw          | rw |
| 15           | 14 | 13           | 12 | 11           | 10 | 9            | 8  | 7            | 6  | 5            | 4  | 3           | 2  | 1           | 0  |
| MODER7[1:0]  |    | MODER6[1:0]  |    | MODER5[1:0]  |    | MODER4[1:0]  |    | MODER3[1:0]  |    | MODER2[1:0]  |    | MODER1[1:0] |    | MODER0[1:0] |    |
| rw           | rw | rw           | rw | rw           | rw | rw           | rw | rw           | rw | rw           | rw | rw          | rw | rw          | rw |

Bits 2y:2y+1 **MODERy[1:0]**: Port x configuration bits (y = 0..15)

These bits are written by software to configure the I/O direction mode.

00: Input (reset state)

01: General purpose output mode

10: Alternate function mode

11: Analog mode

- 1 Clear bits 24-25: `GPIO->MODER &= ~(int32_t)(3 << 24);`
- 2 Set bit 24: `GPIO->MODER |= (1 << 24);`

# Usare gli SFR

Each port x has 11 SFRs:

- **MODER**: configures each bit as input or output or other
- **OSPEEDR**: configures the maximum frequency of an output pin
- **PUPDR**: configures the internal pull-up or pull-down register
- **IDR**: the input data register
- **ODR**: the output data register
- **BSRR**: the bit set/reset register
- **AFRL, AFRH**: alternate function configuration registers
- **LCKR**: the bit lock register
- **OTYPER**: output type configuration (push-pull or open-drain)

Accessing is made:

- By using the predefined **structure pointers**: **GPIOA, GPIOB, GPIOC**
- By accessing the SFR as the structure pointer field: **GPIOA->ODR**

## MODER

|              |     |              |     |              |     |              |     |              |     |              |     |             |     |             |     |
|--------------|-----|--------------|-----|--------------|-----|--------------|-----|--------------|-----|--------------|-----|-------------|-----|-------------|-----|
| 31           | 30  | 29           | 28  | 27           | 26  | 25           | 24  | 23           | 22  | 21           | 20  | 19          | 18  | 17          | 16  |
| MODER15[1:0] |     | MODER14[1:0] |     | MODER13[1:0] |     | MODER12[1:0] |     | MODER11[1:0] |     | MODER10[1:0] |     | MODER9[1:0] |     | MODER8[1:0] |     |
| r/w          | r/w | r/w          | r/w | r/w          | r/w | r/w          | r/w | r/w          | r/w | r/w          | r/w | r/w         | r/w | r/w         | r/w |
| 15           | 14  | 13           | 12  | 11           | 10  | 9            | 8   | 7            | 6   | 5            | 4   | 3           | 2   | 1           | 0   |
| MODER7[1:0]  |     | MODER6[1:0]  |     | MODER5[1:0]  |     | MODER4[1:0]  |     | MODER3[1:0]  |     | MODER2[1:0]  |     | MODER1[1:0] |     | MODER0[1:0] |     |
| r/w          | r/w | r/w          | r/w | r/w          | r/w | r/w          | r/w | r/w          | r/w | r/w          | r/w | r/w         | r/w | r/w         | r/w |

- **MODER** allows a programmer to define the functionality of a GPIO pin
- Each pin has **2 bits** that permits the following configurations:
  - **00**: Input
  - **01**: Output
  - **10**: Alternate Function
  - **11**: Analog

# OTYPER

|      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |
|------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|
| 31   | 30   | 29   | 28   | 27   | 26   | 25   | 24   | 23   | 22   | 21   | 20   | 19   | 18   | 17   | 16   |
| Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. |
| 15   | 14   | 13   | 12   | 11   | 10   | 9    | 8    | 7    | 6    | 5    | 4    | 3    | 2    | 1    | 0    |
| OT15 | OT14 | OT13 | OT12 | OT11 | OT10 | OT9  | OT8  | OT7  | OT6  | OT5  | OT4  | OT3  | OT2  | OT1  | OT0  |
| rW   | rW   | rW   | rW   | rW   | rW   | rW   | rW   | rW   | rW   | rW   | rW   | rW   | rW   | rW   | rW   |

- **OTYPER** allows a programmer to configure the output stage of an output GPIO pin
- Each pin has **1 bits** that permits the following configurations:
  - **0**: Push-pull
  - **1**: Open Drain

# OSPEEDR

|                    |                    |                    |                    |                    |                    |                   |                   |    |    |    |    |    |    |    |    |
|--------------------|--------------------|--------------------|--------------------|--------------------|--------------------|-------------------|-------------------|----|----|----|----|----|----|----|----|
| 31                 | 30                 | 29                 | 28                 | 27                 | 26                 | 25                | 24                | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| OSPEEDR15<br>[1:0] | OSPEEDR14<br>[1:0] | OSPEEDR13<br>[1:0] | OSPEEDR12<br>[1:0] | OSPEEDR11<br>[1:0] | OSPEEDR10<br>[1:0] | OSPEEDR9<br>[1:0] | OSPEEDR8<br>[1:0] |    |    |    |    |    |    |    |    |
| rW                 | rW                 | rW                 | rW                 | rW                 | rW                 | rW                | rW                | rW | rW | rW | rW | rW | rW | rW | rW |
| 15                 | 14                 | 13                 | 12                 | 11                 | 10                 | 9                 | 8                 | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| OSPEEDR7<br>[1:0]  | OSPEEDR6<br>[1:0]  | OSPEEDR5<br>[1:0]  | OSPEEDR4<br>[1:0]  | OSPEEDR3<br>[1:0]  | OSPEEDR2<br>[1:0]  | OSPEEDR1<br>[1:0] | OSPEEDR0<br>[1:0] |    |    |    |    |    |    |    |    |
| rW                 | rW                 | rW                 | rW                 | rW                 | rW                 | rW                | rW                | rW | rW | rW | rW | rW | rW | rW | rW |

- **OSPEEDR** allows a programmer to define the *speed* of an output GPIO pin
- Each pin has **2 bits** that permits the following configurations:
  - **x0**: Low Speed
  - **01**: Medium Speed
  - **11**: High Speed

# PUPDR

|              |     |              |     |              |     |              |     |              |     |              |     |             |     |             |     |
|--------------|-----|--------------|-----|--------------|-----|--------------|-----|--------------|-----|--------------|-----|-------------|-----|-------------|-----|
| 31           | 30  | 29           | 28  | 27           | 26  | 25           | 24  | 23           | 22  | 21           | 20  | 19          | 18  | 17          | 16  |
| PUPDR15[1:0] |     | PUPDR14[1:0] |     | PUPDR13[1:0] |     | PUPDR12[1:0] |     | PUPDR11[1:0] |     | PUPDR10[1:0] |     | PUPDR9[1:0] |     | PUPDR8[1:0] |     |
| r/w          | r/w | r/w          | r/w | r/w          | r/w | r/w          | r/w | r/w          | r/w | r/w          | r/w | r/w         | r/w | r/w         | r/w |
| 15           | 14  | 13           | 12  | 11           | 10  | 9            | 8   | 7            | 6   | 5            | 4   | 3           | 2   | 1           | 0   |
| PUPDR7[1:0]  |     | PUPDR6[1:0]  |     | PUPDR5[1:0]  |     | PUPDR4[1:0]  |     | PUPDR3[1:0]  |     | PUPDR2[1:0]  |     | PUPDR1[1:0] |     | PUPDR0[1:0] |     |
| r/w          | r/w | r/w          | r/w | r/w          | r/w | r/w          | r/w | r/w          | r/w | r/w          | r/w | r/w         | r/w | r/w         | r/w |

- **PUPDR** defines the presence of a pull-up or pull-down resistor (or none) at the GPIO pin
- Each pin has **2 bits** that permits the following configurations:
  - **00**: No pull-up/pull-down
    - When input is **floating**, state is **unknown**
  - **01**: Pull-up
    - When input is **floating**, state is **forced to "1"**
  - **10**: Pull-down
    - When input is **floating**, state is **forced to "0"**

# IDR/ODR

|       |       |       |       |       |       |      |      |      |      |      |      |      |      |      |      |
|-------|-------|-------|-------|-------|-------|------|------|------|------|------|------|------|------|------|------|
| 31    | 30    | 29    | 28    | 27    | 26    | 25   | 24   | 23   | 22   | 21   | 20   | 19   | 18   | 17   | 16   |
| Res.  | Res.  | Res.  | Res.  | Res.  | Res.  | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. |
|       |       |       |       |       |       |      |      |      |      |      |      |      |      |      |      |
| 15    | 14    | 13    | 12    | 11    | 10    | 9    | 8    | 7    | 6    | 5    | 4    | 3    | 2    | 1    | 0    |
| IDR15 | IDR14 | IDR13 | IDR12 | IDR11 | IDR10 | IDR9 | IDR8 | IDR7 | IDR6 | IDR5 | IDR4 | IDR3 | IDR2 | IDR1 | IDR0 |
| r     | r     | r     | r     | r     | r     | r    | r    | r    | r    | r    | r    | r    | r    | r    | r    |

|       |       |       |       |       |       |      |      |      |      |      |      |      |      |      |      |
|-------|-------|-------|-------|-------|-------|------|------|------|------|------|------|------|------|------|------|
| 31    | 30    | 29    | 28    | 27    | 26    | 25   | 24   | 23   | 22   | 21   | 20   | 19   | 18   | 17   | 16   |
| Res.  | Res.  | Res.  | Res.  | Res.  | Res.  | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. |
|       |       |       |       |       |       |      |      |      |      |      |      |      |      |      |      |
| 15    | 14    | 13    | 12    | 11    | 10    | 9    | 8    | 7    | 6    | 5    | 4    | 3    | 2    | 1    | 0    |
| ODR15 | ODR14 | ODR13 | ODR12 | ODR11 | ODR10 | ODR9 | ODR8 | ODR7 | ODR6 | ODR5 | ODR4 | ODR3 | ODR2 | ODR1 | ODR0 |
| r/w   | r/w   | r/w   | r/w   | r/w   | r/w   | r/w  | r/w  | r/w  | r/w  | r/w  | r/w  | r/w  | r/w  | r/w  | r/w  |

- Data Input/Output is performed through the **IDR** and **ODR** registers
- Each pin is mapped to the specific bit, so only 16 bits are used in the registers
- Bit set/reset and check operations are performed through logical mask operations

# BSRR

|      |      |      |      |      |      |     |     |     |     |     |     |     |     |     |     |
|------|------|------|------|------|------|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 31   | 30   | 29   | 28   | 27   | 26   | 25  | 24  | 23  | 22  | 21  | 20  | 19  | 18  | 17  | 16  |
| BR15 | BR14 | BR13 | BR12 | BR11 | BR10 | BR9 | BR8 | BR7 | BR6 | BR5 | BR4 | BR3 | BR2 | BR1 | BR0 |
| w    | w    | w    | w    | w    | w    | w   | w   | w   | w   | w   | w   | w   | w   | w   | w   |
| 15   | 14   | 13   | 12   | 11   | 10   | 9   | 8   | 7   | 6   | 5   | 4   | 3   | 2   | 1   | 0   |
| BS15 | BS14 | BS13 | BS12 | BS11 | BS10 | BS9 | BS8 | BS7 | BS6 | BS5 | BS4 | BS3 | BS2 | BS1 | BS0 |
| w    | w    | w    | w    | w    | w    | w   | w   | w   | w   | w   | w   | w   | w   | w   | w   |

- Single-bit data output (set or reset) can be performed through the **BSRR** register
- The register has two parts: **set part** and **reset part**
- To **set a pin**, a "1" must be written in the correspondent *set part*
- To **reset a pin**, a "1" must be written in the correspondent *reset part*

# BRR

|      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |
|------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|
| 31   | 30   | 29   | 28   | 27   | 26   | 25   | 24   | 23   | 22   | 21   | 20   | 19   | 18   | 17   | 16   |
| Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. |
|      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |
| 15   | 14   | 13   | 12   | 11   | 10   | 9    | 8    | 7    | 6    | 5    | 4    | 3    | 2    | 1    | 0    |
| BR15 | BR14 | BR13 | BR12 | BR11 | BR10 | BR9  | BR8  | BR7  | BR6  | BR5  | BR4  | BR3  | BR2  | BR1  | BR0  |
| w    | w    | w    | w    | w    | w    | w    | w    | w    | w    | w    | w    | w    | w    | w    | w    |

- Single-bit data reset can be also performed through the **BRR** register
- To **reset a pin**, a "1" must be written in the correspondent bit

Premere un bottone e accendere un led:  
situazione classica

```
#include "stm32_unict_lib.h"

int main()
{
    // pushbutton on PB10; LED on PB8

    // initialize ports
    GPIO_init(GPIOB);

    // configure pin PB10 as input
    GPIO_config_input(GPIOB, 10);

    // configure pin PB8 as output
    GPIO_config_output(GPIOB, 8);

    // infinite loop
    for (;;) {
        int pinval = GPIO_read(GPIOB, 10);
        GPIO_write(GPIOB, 8, !pinval);
    }
}
```

Con operazioni bitwise:

Let's replace input reading function with SFR

```
#include "stm32_unict_lib.h"

int main()
{
    // pushbutton on PB10; LED on PB8

    // initialize ports
    GPIO_init(GPIOB);

    // configure pin PB10 as input
    GPIO_config_input(GPIOB, 10);

    // configure pin PB8 as output
    GPIO_config_output(GPIOB, 8);

    // infinite loop
    for (;;) {
        int pinval = (GPIOB->IDR & (1 << 10)) != 0;
        /* pinval is "1" when pushbutton is released */
        /* pinval is "0" when pushbutton is pressed */
        GPIO_write(GPIOB, 8, !pinval);
    }
}
```

Let's replace output writing function with SFR

```
#include "stm32_unict_lib.h"

int main()
{
    // pushbutton on PB10; LED on PB8

    // initialize ports
    GPIO_init(GPIOB);

    // configure pin PB10 as input
    GPIO_config_input(GPIOB, 10);

    // configure pin PB8 as output
    GPIO_config_output(GPIOB, 8);

    // infinite loop
    for (;;) {
        int pinval = (GPIOB->IDR & (1 << 10)) != 0;
        /* pinval is "1" when pushbutton is released */
        /* pinval is "0" when pushbutton is pressed */
        GPIOB->ODR = (GPIOB->ODR & ~(int32_t)0x100) | (!pinval << 8);
    }
}
```

# Funzionalità aggiuntive del timer

I timer hanno degli "slave circuits" che possono essere usati per:

- Misurare il periodo o la pulsazione di un segnale in input;
- Generare un particolare segnale periodico;

Ogni timer del nostro microcontrollore ha 4 "slave circuits" che possono essere collegati a più di una porta GPIO. Ogni canale ha inoltre un registro CCR.

## Input Capture

Quando un canale del timer è configurato come "Input Capture", anche il pin della porta GPIO a cui è collegato deve essere impostato come input: a questo punto, quando il circuito rileva un rising edge o un falling edge (a seconda di come è configurato), GPIO il valore dell'input nel registro CCR e un segnale IRQ viene generato.

## funzioni:

### • Configure the channel:

```
void TIM_config_IC(TIM_TypeDef * tim,  
int channel, IC_Mode mode)
```

- **tim**, the Timer
- **channel**, the slave channel, from 1 to 4
- **mode**, the capture mode, **IC\_RISING\_EDGE**, **IC\_FALLING\_EDGE**, **IC\_BOTH\_EDGES**

### • Check a capture event:

```
int TIM_cc_check(TIM_TypeDef * tim, int channel)
```

- **tim**, the Timer
- **channel**, the slave channel, from 1 to 4
- returns "true" or "false"

### • Clear a capture event:

```
void TIM_cc_clear(TIM_TypeDef * tim, int channel)
```

- **tim**, the Timer
- **channel**, the slave channel, from 1 to 4

## Esempio:

- We want to measure the **reaction time** of a person
  - We lit a LED and wait for the person to press a button
  - We measure the **time** between LED lighting and button press
  - After 5 secs, we notify a **timeout**
- 
- We want to measure the **reaction time** of a person
    - We use a timer with a “capture” function
  - We lit a LED
    - We set to “1” the GPIO and reset the timer
  - and wait for the person to press a button
    - We wait for capture interrupt
  - We measure the **time** between LED lighting and button press
    - We read the capture-compare register
  - After 5 secs, we notify a **timeout**
    - We use the “period match” (ARR)

```
#include "stm32_unict_lib.h"

int main()
{
    DISPLAY_init();
    GPIO_init(GPIOB);
    GPIO_config_output(GPIOB, 0); // the LED
    GPIO_config_alternate(GPIOB, 4, 2); // button "Y" as AF2 --> TIM3,CH1

    TIM_init(TIM3);
    TIM_config_timebase(TIM3, 8400, 50000); // prescaler at 100us timeout of 5 secs
    TIM_config_IC(TIM3, 1, IC_FALLING_EDGE); // capture on falling-edge (pushbutton)
    TIM_set(TIM3, 0);
    TIM_on(TIM3);
    TIM_enable_irq(TIM3, IRQ_UPDATE | IRQ_CC_1);

    ...
}
```

```
...
for (;;) {
    delay_ms(3000); // wait 3 seconds to start the game
    TIM_set(TIM3, 0); // clear timer and flags
    measure_done = 0;
    timeout = 0;
    GPIO_write(GPIOB, 0, 1); // turn on LED
    DISPLAY_puts(0, "___");
    while (!measure_done && !timeout) ; // wait for capture or timeout
    if (measure_done) {
        // capture OK!
        int ms = TIM3->CCR1 / 10;
        char s[4];
        sprintf(s, "%4d", ms);
        DISPLAY_puts(0, s);
    }
    else {
        // timeout
        DISPLAY_puts(0, "tout");
    }
}
}
...

```

```
void TIM3_IRQHandler(void)
{
    if (TIM_update_check(TIM3)) {
        // period match, timeout
        GPIO_write(GPIOB, 0, 0);
        timeout = 1;
        TIM_update_clear(TIM3);
    }
    if (TIM_cc_check(TIM3, 1)) {
        // capture
        GPIO_write(GPIOB, 0, 0);
        measure_done = 1;
        TIM_cc_clear(TIM3, 1);
    }
}

```

# PWM Output

Un canale del timer può essere configurato per generare un PWM signal con una specifica frequenza e con uno specifico ciclo di lavoro.

Il periodo, la frequenza, è espresso con il registro ARR del timer.  
Il ciclo di lavoro (duty cycle), rappresenta la durata della parte positiva del segnale, il cui valore è assegnato al registro CCR.

## funzioni:

- To configure a channel in PWM mode the following function is provided:  
`void TIM_config_PWM(TIM_TypeDef * tim, int channel)`
  - **tim**, the Timer
  - **channel**, the slave channel, from 1 to 4
- The **period** is set as the "autoreload" parameter in the `TIM_config_timebase` function
- The **duty cycle** is set by assigning the **CCRy** register

## Controllare l'intensità del segnale usando PWM

```
#include <stdio.h>
#include "stm32_unict_lib.h"

#define PERIOD_PWM 1000

int main(void)
{
    DISPLAY_init();
    GPIO_init(GPIOB);
    GPIO_config_alternate(GPIOB, 0, 2); // AF2 --> TIM3_CH3

    ADC_init(ADC1, ADC_RES_8, ADC_ALIGN_RIGHT);
    ADC_channel_config(ADC1, GPIOC, 0, 10);
    ADC_on(ADC1);
    ADC_sample_channel(ADC1, 10);

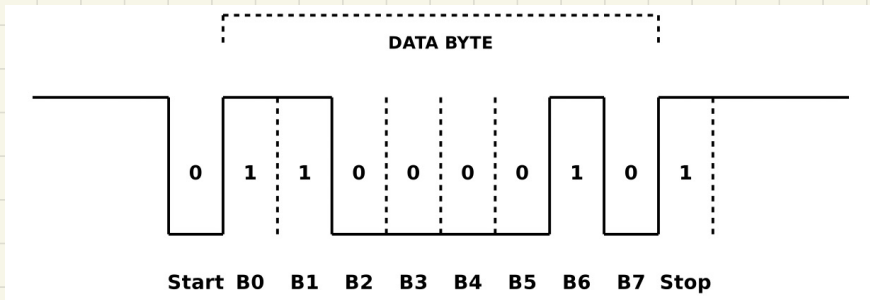
    TIM_init(TIM3);
    TIM_config_timebase(TIM3, 420, PERIOD_PWM);
    TIM_set(TIM3, 0);
    TIM_config_PWM(TIM3, 3); // channel 3
    TIM_on(TIM3);

    for (;;) {
        ADC_start(ADC1);
        while (!ADC_completed(ADC1)) {}
        int t_on = (ADC_read(ADC1) * 1000) / 255;
        TIM3->CCR3 = t_on;
        char s[4];
        sprintf(s, "%4d", t_on);
        DISPLAY_puts(0, s);
    }
    return 0;
}
```

# UART

UART sta per Universal Asynchronous Receiver Transmitter ed è una periferica per la comunicazione punto-punto tra due dispositivi. La comunicazione avviene un bit alla volta attraverso 2 pin: RX e TX, il primo per la ricezione e il secondo per la trasmissione.

Quando non c'è trasmissione, il pin è settato ad 1. Quando la trasmissione comincia, è trasmesso prima il bit 0, chiamato "start bit", poi avviene la trasmissione dei dati a partire dal bit meno significativo (con in aggiunta un eventuale bit di parità), ed infine viene trasmesso il bit 1, chiamato anche "stop bit".



Nella comunicazione, il bit di parità può essere settato come:

- NONE: il bit di parità non viene trasmesso;
- MARK: il bit di parità trasmesso è "1";
- SPACE: il bit di parità trasmesso è "0";
- ODD: il numero di 1 deve essere dispari;
- EVEN: il numero di 1 deve essere pari;

Nell'hardware UART bisogna impostare i seguenti parametri:

- La velocità di trasmissione in bit per secondo (bps);
- Numero di bit per carattere (di solito 8);
- Presenza/assenza del bit di parità (di solito assente);
- Numero di "stop bit" (di solito 1);

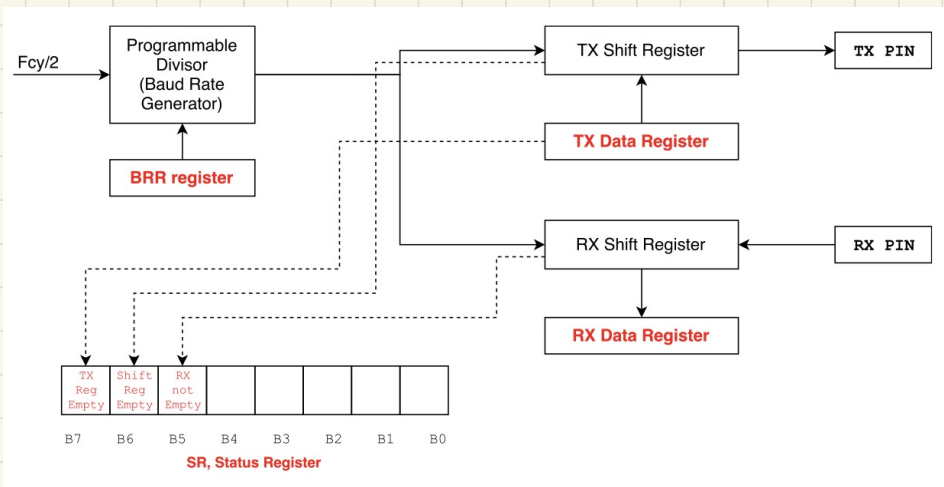
Lo schema di funzionamento dell'UART è diviso in 3 blocchi;

- BAUD RATE GENERATOR;
- RECEIVER CIRCUIT;
- TRANSMITTER CIRCUIT;

Il Baud Rate Generator è responsabile della generazione del clock per la trasmissione dei dati ed è un divisore programmabile che parte dalla metà della frequenza di clock della CPU (nel nostro caso  $\frac{80}{2} = 40 \text{ kHz}$ ).

Il Receiver Circuit è responsabile della ricezione dei dati. Ha due registri: RX Shift Register e RX Data Register. Quando un byte è completato, il contenuto dello Shift Register è copiato nel Data Register e nello Status Register viene settato il bit "RX Register not Empty".

Il Transmitter Circuit è responsabile della trasmissione dei dati. Ha due registri: TX Shift Register e TX Data Register. I bit da trasmettere sono caricati nel Data Register, poi vengono copiati nello Shift Register e nello Status Register viene settato il bit "TX Register Empty". Una volta trasmessi i dati viene settato nello Status Register il bit "TX Shift Register Empty".



Ogni cambiamento nello Status Register può essere utilizzato per generare un IRQ, così da partire la comunicazione attraverso l'attivazione di Routine di interrupt.

## funzioni;

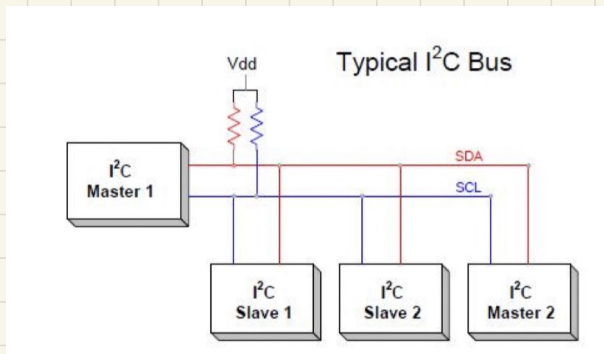
- Configure the USART2 and set baud rate to 115200 bps:  
`void CONSOLE_init(void);`
- Output messages to USART2:  
`int printf(...);`
- Check if a data byte has been received:  
`int kbhit(void);`
- Read/Wait for a new data byte:  
`char readchar(void);`

# I<sup>2</sup>C Bus Interface

IIC o I<sup>2</sup>C sta per Inter-Integrated Circuit Bus ed è un bus di comunicazione progettato per interconnettere i circuiti integrati che stanno sulla stessa scheda.

Un network I<sup>2</sup>C è già composto: c'è il dispositivo "master" che è la testa del bus ed è responsabile dell'inizio e della fine delle comunicazioni e i dispositivi "slave" che hanno il compito di eseguire le istruzioni del master.

Inoltre, i dispositivi sono collegati da due fili, entrambi seriali: l'SDA, bidirezionale, dove viaggiano i dati, e l'SCL, attraverso cui il master detta il tempo della trasmissione agli slave ed è unidirezionale.

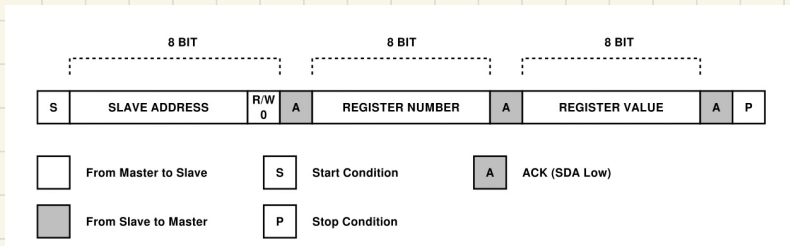


Ogni dispositivo slave ha un indirizzo e vari registri che possono essere letti o scritti. L'indirizzo è composto da 7 bit + 1 (R/S), che serve a specificare la direzione del trasferimento dati: write-to-slave o read-from-slave. Sia la lettura che la scrittura su un registro richiedono l'esecuzione di una serie di istruzioni.

## Write Transaction

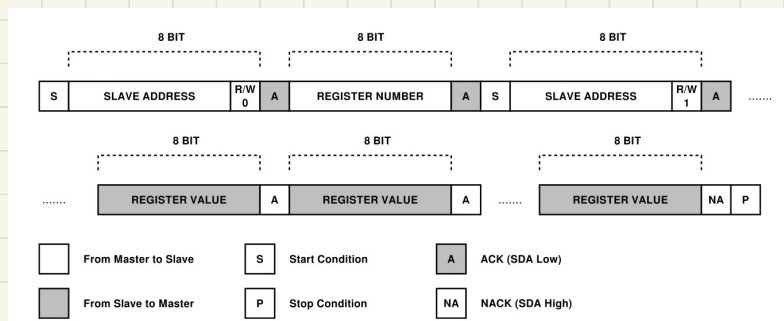
- 1) Il Master invia la "Start Condition";
- 2) Il Master invia l'indirizzo del dispositivo con 10 posto a "0";
- 3) Il Master invia il numero del registro che deve essere scritto;
- 4) Il Master invia i dati che devono essere scritti nel registro;
- 5) Il Master invia la "Stop Condition";

Per ogni byte scritto, lo slave invia un "ack".



## Read Transaction

- 1) Il Master invia la "Start Condition";
- 2) Il Master invia l'indirizzo del dispositivo con 10 posto a "0";
- 3) Il Master invia il numero del registro che deve essere letto;
- 4) Il Master invia una nuova "Start Condition";
- 5) Il master invia l'indirizzo del dispositivo con 10 posto a "1";
- 6) Lo slave invia i dati del registro che deve essere letto;
- 7) Il Master invia la "Stop Condition";



# funzioni:

- **Initialize the I2C interface:**

```
void I2C_init(I2C_TypeDef * port, int speed);
```

- **port**, the interface (usually I2C1)
- **speed**, clock speed (e.g. 100000, 400000)

- **Write a single register:**

```
void I2C_write_register(I2C_TypeDef * port,  
                       short addr, short reg_addr,  
                       short reg_val);
```

- **port**, the interface (usually I2C1)
- **addr**, device address
- **reg\_addr**, register address
- **reg\_val**, register value

- **Read a single register:**

```
void I2C_read_register(I2C_TypeDef * port,  
                      short addr, short reg_addr,  
                      short * reg_val);
```

- **port**, the interface (usually I2C1)
- **addr**, device address
- **reg\_addr**, register address
- **reg\_val**, pointer to the variable that will receive register value

- **Write a set of registers:**

```
void I2C_write_buffer(I2C_TypeDef * port,  
                     short addr, short reg_addr,  
                     unsigned char * data, int len);
```

- **port**, the interface (usually I2C1)
- **addr**, device address
- **reg\_addr**, register address
- **data**, the buffer holding register values
- **len**, the buffer length (number of registers)

- **Read a set of registers:**

```
void I2C_read_buffer(I2C_TypeDef * port,  
                    short addr, short reg_addr,  
                    unsigned char * data, int len);
```

- **port**, the interface (usually I2C1)
- **addr**, device address
- **reg\_addr**, register address
- **data**, the buffer that will receive register values
- **len**, the buffer length (number of registers)

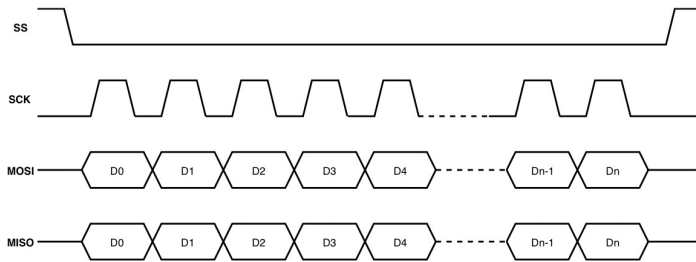
# Serial Peripheral Interface (SPI)

L'SPI è un hardware di comunicazione progettato per connettere circuiti integrati che stanno sulla stessa scheda.

L'SPI è un'interfaccia di tipo Master-Slave e ha 4 fili unidirezionali:

- SCK: Master-to-Slave, il clock generato dal Master;
- MOSI: Master-to-Slave, i dati trasmessi dal Master allo Slave;
- MISO: Slave-to-Master, i dati trasmessi dallo Slave al Master;
- SS: Slave-to-Master, il segnale logico usato per selezionare e "svegliare" il dispositivo Slave;

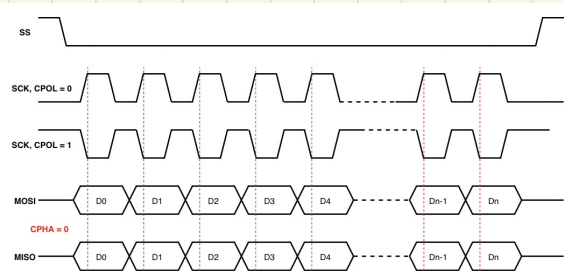
Una trasmissione di dati comincia sempre ponendo a "0" la linea SS. Dopo di che il Master comincia a generare il clock e ad ogni "tic-tac" del clock si trasmette un bit. Quando la trasmissione termina, il Master pone a "1" la linea SS.



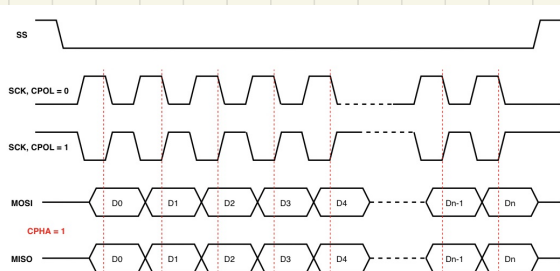
A seconda di come impostiamo il clock, otteniamo 2 schemi di comunicazione diversi.

Con la Clock-Polarity, possiamo impostare lo stato logico iniziale del clock.

Con il Clock-Phase possiamo impostare il clock usato per la trasmissione (se il primo o il secondo).



**CPHA = 0**, data is sampled in the first edge of the clock



**CPHA = 1**, data is sampled in the second edge of the clock

## Differenza I<sup>2</sup>C e SPI

### Fili:

- I<sup>2</sup>C: 2
- SPI: 4

### Aspetti Elettrici:

- I<sup>2</sup>C: ha anche un filo unidirezionale
- SPI: ha solo fili unidirezionali

### Velocità:

- I<sup>2</sup>C: 400 kbps
- SPI: Teoricamente nessun limite, consigliato 1 Mbps

### Protocolli e Indirizzi:

- I<sup>2</sup>C: Ha uno standard per gli indirizzi dei dispositivi e le funzionalità sono mappate nei registri
- SPI: non ha uno standard per gli indirizzi, ogni dispositivo ha un dato format

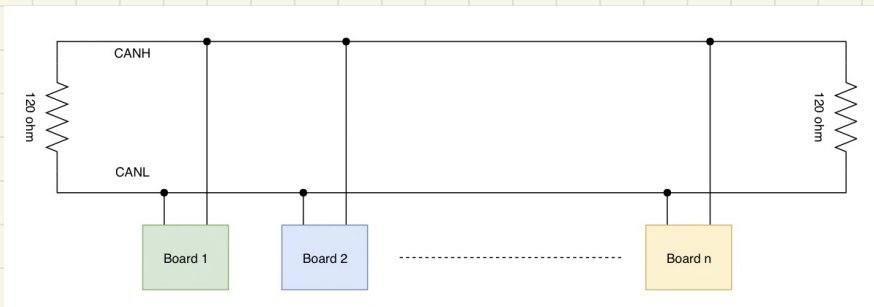
### Numero di dispositivi supportati:

- I<sup>2</sup>C: È un bus, quindi in teoria niente limiti
- SPI: i limiti sono definiti da (numero di fili (in caso di connessione parallela), o dalla latenza (in caso di connessione daisy-chain))

# Controller Area Network Interface (CAN)

L'interfaccia CAN è pensata per la comunicazione e l'interconnessione tra più microcontrollori.

Dal punto di vista elettrico utilizza una coppia di BUS e i device sono collegati ad esso per mezzo di due segnali, CANH (high) e CANL (low).



Tutti i dispositivi sono "pari", cioè non esiste una relazione "master-slave" e ogni dispositivo può decidere di iniziare la comunicazione in qualsiasi momento.

I bit trasmessi sono:

- 1, detto bit recessivo;
- 0, detto bit dominante;

Ogni dispositivo è in grado di ascoltare i bit trasmessi dagli altri, quindi quello che trasmette il bit dominante "vince", mentre gli altri sospendono la trasmissione.

Un pacchetto nel circuito CAN è formato da 3 parti principali:

- Il Payload;
- CAN-ID, usato per identificare i dati e la priorità;
- Bit di controllo;

I dispositivi gestiscono la comunicazione tramite il meccanismo "publisher - subscriber".

Dunque un nodo, il publisher, invia i dati usando un certo CAN-ID e il pacchetto viene inviato in broadcast a tutti i nodi.

Dopo di che, ogni nodo interessato a quel CAN-ID, i subscriber, "catturano" i dati e li mandano al software.

Quando un dispositivo invia i dati, mette un bit recessivo (1) nello slot del ACK. Se un dispositivo è interessato al pacchetto, inserisce un bit dominante (0) nello slot dell'ACK. In questo modo il sender sa se il pacchetto è stato ricevuto o meno. Se nessuno ha ricevuto il pacchetto, il sender ci riprova al massimo altre 127 volte.

